IBM FileNet P8 V5.5.8

Step-by-Step Hands-On Lab Guide

Day 9 – API Integration & Automation

REST API | Java CEI API | Event Subscriptions | Workflow Automation

Author	Course Day	Lab Count	Total Duration
Rajesh Pasham	**Day 9 of 10**	**4 Labs**	**~3 Hours**

LAB-09-01	LAB-09-02	LAB-09-03	LAB-09-04
Environment & Prerequisites Setup *30 min | Java, curl, Postman, CE JAR files*	**Upload Document via REST API** *45 min | CMIS Browser Binding + curl/Postman*	**Java CEI API – CRUD Operations** *60 min | Connect, create, update, search*	**Event Subscription & Workflow Trigger** *45 min | OnCreate handler + PE REST launch*

LAB-09-01 | Environment & Prerequisites Setup

Duration: 30 Minutes | Verify lab environment and install all required tools

NOTE: START STATE: IBM FileNet P8 V5.5.8 is installed. Content Platform Engine (CE) is running on WAS at localhost:9080. TrainingOS Object Store exists. Process Engine (PE) V5.5.8 is installed and PE_ConnectionPoint is configured. ACCE is accessible at http://localhost:9080/acce.

Prerequisites Checklist

All labs in Day 9 depend on the following components being available. Verify each item before proceeding.

STEP 1 — Verify Content Engine is Running (5 min)

1. Open a browser and navigate to: http://localhost:9080/acce
2. Log in with: ceadmin / ceadmin
3. Verify you can see P8Domain > TrainingOS in the left navigation tree
4. Navigate to TrainingOS > Browse > Root Folder and confirm the folder tree loads

	VERIFICATION CHECKLIST
[ ]	ACCE loads successfully at http://localhost:9080/acce
[ ]	P8Domain and TrainingOS are visible in the navigation tree
[ ]	Root Folder browsable with no errors

STEP 2 — Verify CMIS Endpoint is Accessible (5 min)

5. Open a new browser tab and navigate to:

CMIS Browser URL	`http://localhost:9080/fncmis/browser?cmisselector=repositories`

6. You should see a JSON response listing available repositories
7. Locate the repositoryId value for TrainingOS — it will be a GUID like {XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX}
8. Copy this repositoryId — you will need it throughout Lab 09-02

TIP: If the CMIS endpoint returns a 404, verify that the fncmis.war application is deployed in WAS Admin Console under Applications > WebSphere Enterprise Applications. It should show status 'Started'.

	VERIFICATION CHECKLIST
[ ]	http://localhost:9080/fncmis/browser returns valid JSON
[ ]	repositoryId GUID for TrainingOS is identified and copied

STEP — Verify curl is Available (3 min)

3

9. Open a Command Prompt (Windows) or terminal
10. Run the following command to verify curl is installed:

```
curl --version
```

11. Expected output: curl 7.x.x or higher

WARNING: On Windows Server 2019+, curl.exe is included by default. On older Windows Server versions, download curl from https://curl.se/windows/ and add it to PATH. For lab environments use the curl bundled with Git for Windows if available.

	VERIFICATION CHECKLIST
[ ]	curl --version returns a version number without errors

STEP 4 Locate and Verify FileNet JAR Files

5 min

The Java CEI API labs (Lab 09-03) require the FileNet JAR files from the CE installation. Verify they exist:

```
# Windows - verify JAR files exist
dir "C:\IBM\FileNet\ContentEngine\lib\Jace.jar"
dir "C:\IBM\FileNet\ContentEngine\lib\log4j.jar"
dir "C:\IBM\FileNet\ContentEngine\lib\stax-api.jar"
```

NOTE: If the ContentEngine directory is at a different path, check the FileNet installation log or ACCE > Admin > System Properties for the CE install path. The standard path for V5.5.8 on Windows is C:\IBM\FileNet\ContentEngine.

	VERIFICATION CHECKLIST
[ ]	Jace.jar found at C:\IBM\FileNet\ContentEngine\lib\Jace.jar
[ ]	log4j.jar found at C:\IBM\FileNet\ContentEngine\lib\log4j.jar
[ ]	stax-api.jar found at C:\IBM\FileNet\ContentEngine\lib\stax-api.jar

STEP 5 Create Lab Working Directory and Test File

5 min

12. Create a working directory for all Day 9 lab files:

```
mkdir C:\Day9Labs
mkdir C:\Day9Labs\java
mkdir C:\Day9Labs\rest
```

13. Create a test document for upload labs:

```
echo This is a FileNet API test document. Lab Day 9. >
C:\Day9Labs\rest\test_document.txt
```

```
echo IBM FileNet P8 V5.5.8 - API Integration Lab >>
C:\Day9Labs\rest\test_document.txt
```

14. Verify the file was created:

```
type C:\Day9Labs\rest\test_document.txt
```

	VERIFICATION CHECKLIST
[ ]	Directory C:\Day9Labs\rest exists
[ ]	C:\Day9Labs\rest\test_document.txt created with content
[ ]	type command shows the file contents correctly

STEP 6 — Create a Target Folder in FileNet for Lab Documents — 7 min

15. In ACCE, navigate to: Object Stores > TrainingOS > Browse > Root Folder
16. Right-click Root Folder > New Folder

Folder Name	API_Labs
Folder Class	Folder
Parent Folder	/ (Root Folder)

17. Click Finish. The API_Labs folder should appear under Root Folder.
18. Create a sub-folder inside API_Labs:

Folder Name	REST_Uploads
Parent Folder	/API_Labs

19. Repeat to create another sub-folder: Java_Uploads under /API_Labs

[SCREENSHOT]
ACCE Browse view showing Root Folder > API_Labs > REST_Uploads and Java_Uploads

	VERIFICATION CHECKLIST
[ ]	/API_Labs folder created in Root Folder
[ ]	/API_Labs/REST_Uploads sub-folder created
[ ]	/API_Labs/Java_Uploads sub-folder created
[ ]	All folders visible in ACCE Browse tree

LAB-09-02 | Upload Document via REST API

Duration: 45 Minutes | Use CMIS Browser Binding and curl to upload documents to FileNet

NOTE: START STATE: Lab-09-01 completed. curl is available. test_document.txt exists at C:\Day9Labs\rest\test_document.txt. API_Labs/REST_Uploads folder exists in TrainingOS. repositoryId GUID was copied from Step 2 of Lab-09-01.

Objective

In this lab you will:

- Retrieve the TrainingOS repository ID and folder structure via CMIS
- Upload a text document to the REST_Uploads folder using curl and CMIS Browser Binding
- Set document metadata (name, description) during upload
- Verify the document appears in ACCE after upload
- Retrieve document properties via REST API using the returned objectId
- Execute a CMIS search query to find the uploaded document

STEP 1 | Get Repository ID and Root Children | 5 min

20. Open a Command Prompt in C:\Day9Labs\rest
21. Run the following command to get all repositories. Save the output:

```
curl -u ceadmin:ceadmin ^
     -H "Accept: application/json" ^
     "http://localhost:9080/fncmis/browser?cmisselector=repositories" > repos.json

type repos.json
```

22. Look for the repositoryId field in the JSON. It will be a GUID:

Expected Field	`repositoryId`
Example Value	`{XXXXXXXX-XXXX-XXXX-XXXX-XXXXXXXXXXXX}`
Store As	`REPO_ID (use in all subsequent commands)`

TIP: On Windows Command Prompt, use ^ for line continuation in curl commands. In PowerShell use ` (backtick). In this lab all examples use Windows CMD ^ syntax.

STEP 2 | Get the API_Labs Folder ID | 8 min

To file a document into a specific folder via CMIS, you need the folder's CMIS objectId OR you can use the folder path directly in the URL. We will use the folder path approach:

23. Replace {REPO_ID} with your actual repository GUID in all commands below
24. Test navigating to the REST_Uploads folder by path:

```
curl -u ceadmin:ceadmin ^
     -H "Accept: application/json" ^
```

```
"http://localhost:9080/fncmis/browser/{REPO_ID}/API_Labs/REST_Uploads?cmisselector=object"
> folder_info.json

type folder_info.json
```

25. The response should show the folder object with its cmis:objectId property
26. Note the cmis:objectId value — this is the folder's FileNet ID (starting with {)

	VERIFICATION CHECKLIST
[ ]	curl returns valid JSON with cmis:objectId for REST_Uploads folder
[ ]	cmis:objectId noted for future use

STEP 3 — Upload a Document to REST_Uploads Folder — 15 min

Use the CMIS createDocument action to upload the test file into the REST_Uploads folder. This is a multipart form POST:

27. Navigate to your working directory:

```
cd C:\Day9Labs\rest
```

28. Run the upload command (replace {REPO_ID} with your actual GUID):

```
curl -u ceadmin:ceadmin ^
     -X POST ^
     -H "Accept: application/json" ^
     -F "cmisaction=createDocument" ^
     -F "propertyId[0]=cmis:name" ^
     -F "propertyValue[0]=REST_API_Lab_Document.txt" ^
     -F "propertyId[1]=cmis:objectTypeId" ^
     -F "propertyValue[1]=cmis:document" ^
     -F "propertyId[2]=cmis:description" ^
     -F "propertyValue[2]=Uploaded via REST API Lab Day 9" ^
     -F "content=@test_document.txt;type=text/plain" ^
     "http://localhost:9080/fncmis/browser/{REPO_ID}/API_Labs/REST_Uploads" ^
     > upload_result.json

type upload_result.json
```

TIP: If you get a 400 Bad Request error, check: (1) the folder path is correct (case-sensitive), (2) the file test_document.txt is in the current directory, (3) the REPO_ID GUID is correct. Add -v to the curl command for verbose output to debug.

29. On success, the response JSON will contain a cmis:objectId for the new document
30. Copy the cmis:objectId value — you will use it in Steps 4 and 5

[SCREENSHOT]
Terminal output showing successful curl upload response with cmis:objectId

	VERIFICATION CHECKLIST
[ ]	curl command executes without error (HTTP 201 Created)
[ ]	upload_result.json contains cmis:objectId for the new document
[ ]	cmis:objectId of uploaded document is noted

STEP 4 Verify Upload in ACCE — 5 min

31. Switch to ACCE in the browser: http://localhost:9080/acce
32. Navigate to: Object Stores > TrainingOS > Browse > API_Labs > REST_Uploads
33. Verify REST_API_Lab_Document.txt appears in the folder
34. Right-click the document > Properties and check:

Document Title	`REST_API_Lab_Document.txt`
Description	`Uploaded via REST API Lab Day 9`
Storage Area	`FS_TrainingStore_01 (or configured default)`

[SCREENSHOT]
ACCE Browse view showing REST_API_Lab_Document.txt in REST_Uploads folder

	VERIFICATION CHECKLIST
[ ]	Document REST_API_Lab_Document.txt is visible in ACCE at /API_Labs/REST_Uploads
[ ]	Document description shows: Uploaded via REST API Lab Day 9
[ ]	Document has binary content (not empty)

STEP 5 Retrieve Document Properties via REST API — 7 min

35. Use the objectId from Step 3 to retrieve the document properties:

```
curl -u ceadmin:ceadmin ^
     -H "Accept: application/json" ^

"http://localhost:9080/fncmis/browser/{REPO_ID}?cmisselector=object&objectId={DOC_OBJECT_I
D}" ^
     > doc_props.json

type doc_props.json
```

36. Review the JSON response. Key properties to identify:

cmis:objectId	`The FileNet GUID of the document`
cmis:name	`Document name (cmis:document type)`
cmis:description	`Description set during upload`
cmis:creationDate	`Timestamp when created`

cmis:contentStreamLength	File size in bytes
cmis:contentStreamMimeType	MIME type (text/plain)

	VERIFICATION CHECKLIST
[ ]	curl retrieves doc_props.json without error
[ ]	cmis:name shows REST_API_Lab_Document.txt
[ ]	cmis:description shows Uploaded via REST API Lab Day 9
[ ]	cmis:contentStreamLength shows a non-zero value

STEP 6 — Search for Uploaded Document via CMIS Query — 5 min

```
curl -u ceadmin:ceadmin ^
     -X POST ^
     -H "Content-Type: application/x-www-form-urlencoded" ^
     -H "Accept: application/json" ^
     --data-urlencode "cmisaction=query" ^
     --data-urlencode "statement=SELECT * FROM cmis:document WHERE cmis:name =
'REST_API_Lab_Document.txt'" ^
     --data-urlencode "maxItems=10" ^
     "http://localhost:9080/fncmis/browser/{REPO_ID}" ^
     > search_result.json

type search_result.json
```

	VERIFICATION CHECKLIST
[ ]	Search returns 1 result matching REST_API_Lab_Document.txt
[ ]	numItems in response is 1
[ ]	cmis:objectId in results matches the ID from Step 3

LAB-09-03 | Java CEI API – CRUD Operations

Duration: 60 Minutes | Connect to CE V5.5.8 via Java MTOM, create/update/search documents

NOTE: START STATE: Lab-09-01 completed. JDK 8 or higher is installed (java -version works). JAR files verified at C:\IBM\FileNet\ContentEngine\lib\. C:\Day9Labs\java directory created.

Objective

In this lab you will:

- Set up a Java project classpath with FileNet V5.5.8 JAR files
- Write and compile Java code to connect to Content Engine via MTOM
- Create a new document in the Java_Uploads folder with metadata
- Update the document's metadata using the CEI API
- Execute a CE SQL search query and print results

STEP 1 | Verify Java Development Kit (JDK) is Installed | 3 min

```
java -version
# Expected: java version "1.8.x" or higher

javac -version
# Expected: javac 1.8.x or higher
```

WARNING: The FileNet P8 V5.5.8 Jace.jar is compiled for Java 8. Use JDK 8 for this lab to avoid compatibility issues. If only JDK 11+ is available, the Java code will still compile but you must add --add-opens flags to suppress module warnings.

	VERIFICATION CHECKLIST
[ ]	java -version returns 1.8.x or higher
[ ]	javac -version returns a version number

STEP 2 | Create the FileNetConnect.java Source File | 5 min

Create the following Java file at C:\Day9Labs\java\FileNetConnect.java using Notepad or any text editor:

```
// FileNetConnect.java - FileNet P8 V5.5.8 Connection Test
// Save as: C:\Day9Labs\java\FileNetConnect.java

import com.filenet.api.core.*;
import com.filenet.api.util.UserContext;
import javax.security.auth.Subject;

public class FileNetConnect {
    public static final String CE_URI = "http://localhost:9080/wsi/FNCEWS40MTOM/";
    public static final String USERNAME = "ceadmin";
    public static final String PASSWORD = "ceadmin";
```

```
    public static final String OS_NAME  = "TrainingOS";

    public static void main(String[] args) throws Exception {
        System.out.println("Connecting to FileNet P8 V5.5.8...");
        Connection conn = Factory.Connection.getConnection(CE_URI);
        Subject subject = UserContext.createSubject(conn, USERNAME, PASSWORD,
"FileNetP8");
        UserContext.get().pushSubject(subject);
        try {
            Domain domain = Factory.Domain.fetchInstance(conn, null, null);
            System.out.println("Connected to domain: " + domain.get_Name());
            ObjectStore os = Factory.ObjectStore.fetchInstance(domain, OS_NAME,
null);
            System.out.println("Object store: " + os.get_Name() + " (" + os.get_Id()
+ ")");
            System.out.println("CONNECTION SUCCESSFUL");
        } finally {
            UserContext.get().popSubject();
        }
    }
}
```

STEP 3 — Compile and Run the Connection Test (8 min)

37. Open a Command Prompt and navigate to the Java lab folder:

```
cd C:\Day9Labs\java
```

38. Compile FileNetConnect.java with the FileNet JAR files on the classpath:

```
javac -cp "C:\IBM\FileNet\ContentEngine\lib\Jace.jar;^
C:\IBM\FileNet\ContentEngine\lib\log4j.jar;^
C:\IBM\FileNet\ContentEngine\lib\stax-api.jar" ^
FileNetConnect.java
```

39. If compilation succeeds (no errors), run the program:

```
java -cp ".;C:\IBM\FileNet\ContentEngine\lib\Jace.jar;^
C:\IBM\FileNet\ContentEngine\lib\log4j.jar;^
C:\IBM\FileNet\ContentEngine\lib\stax-api.jar" ^
FileNetConnect
```

TIP: If you get a ClassNotFoundException for com.filenet.api.core.Factory, ensure Jace.jar is on the classpath. If you get an IIOP connection error, verify the CE URL is http://localhost:9080/wsi/FNCEWS40MTOM/ (MTOM, not IIOP).

	VERIFICATION CHECKLIST
[ ]	FileNetConnect.java compiles without errors
[ ]	Program output shows: Connected to domain: P8Domain
[ ]	Program output shows: Object store: TrainingOS
[ ]	Program output shows: CONNECTION SUCCESSFUL

STEP 4 Create FileNetCRUD.java – Document Create and Update (15 min)

Create C:\Day9Labs\java\FileNetCRUD.java with the following content:

```
// FileNetCRUD.java - Create, Update, Search via CEI API
// FileNet P8 V5.5.8
import com.filenet.api.core.*;
import com.filenet.api.util.UserContext;
import com.filenet.api.property.*;
import com.filenet.api.query.*;
import com.filenet.api.collection.*;
import com.filenet.api.constants.*;
import javax.security.auth.Subject;
import java.io.*;

public class FileNetCRUD {
    static final String CE_URI = "http://localhost:9080/wsi/FNCEWS40MTOM/";
    static final String USERNAME = "ceadmin";
    static final String PASSWORD = "ceadmin";
    static final String OS_NAME  = "TrainingOS";

    public static void main(String[] args) throws Exception {
        Connection conn = Factory.Connection.getConnection(CE_URI);
        Subject subj = UserContext.createSubject(conn, USERNAME, PASSWORD,
"FileNetP8");
        UserContext.get().pushSubject(subj);
        try {
            Domain domain = Factory.Domain.fetchInstance(conn, null, null);
            ObjectStore os = Factory.ObjectStore.fetchInstance(domain, OS_NAME,
null);
            System.out.println("Connected to: " + os.get_Name());

            // ---- CREATE DOCUMENT ----
            String docId = createDocument(os);

            // ---- UPDATE METADATA ----
            updateMetadata(os, docId);

            // ---- SEARCH ----
            searchDocuments(os);

        } finally {
            UserContext.get().popSubject();
        }
    }

    static String createDocument(ObjectStore os) throws Exception {
        // Create test content file
        File f = new File("C:\\Day9Labs\\java\\java_test.txt");
        PrintWriter pw = new PrintWriter(f);
        pw.println("Created by FileNet Java CEI API - Day 9 Lab");
        pw.println("IBM FileNet P8 V5.5.8");
        pw.close();

        Document doc = Factory.Document.createInstance(os, "Document");
```

```
        doc.getProperties().putValue("DocumentTitle", "Java_API_Lab_Document");

        ContentElementList cel = Factory.ContentElement.createList();
        ContentTransfer ct = Factory.ContentTransfer.createInstance();
        ct.setCaptureSource(new FileInputStream(f));
        ct.set_RetrievalName("java_test.txt");
        ct.set_ContentType("text/plain");
        cel.add(ct);
        doc.set_ContentElements(cel);

        doc.checkin(AutoClassify.DO_NOT_AUTO_CLASSIFY, CheckinType.MAJOR_VERSION);
        doc.save(RefreshMode.REFRESH);

        // File into folder
        Folder folder = Factory.Folder.fetchInstance(os, "/API_Labs/Java_Uploads",
null);
        ReferentialContainmentRelationship rcr =
            folder.file(doc, AutoUniqueName.AUTO_UNIQUE, "Java_API_Lab_Document",
                        DefiningObjectStore.TOGETHER_WITH_CONTENT);
        rcr.save(RefreshMode.NO_REFRESH);

        String id = doc.get_Id().toString();
        System.out.println("[CREATE] Document created. ID: " + id);
        return id;
    }

    static void updateMetadata(ObjectStore os, String docId) {
        Document doc = Factory.Document.fetchInstance(os,
            new com.filenet.api.util.Id(docId), null);
        doc.getProperties().putValue("DocumentTitle",
"Java_API_Lab_Document_UPDATED");
        doc.save(RefreshMode.NO_REFRESH);
        System.out.println("[UPDATE] Title updated for ID: " + docId);
    }

    static void searchDocuments(ObjectStore os) {
        SearchSQL sql = new SearchSQL(
            "SELECT Id, DocumentTitle FROM Document WHERE DocumentTitle LIKE
'%Java_API%'");
        SearchScope scope = new SearchScope(os);
        IndependentObjectSet results = scope.fetchObjects(sql, 50, null,
Boolean.TRUE);
        java.util.Iterator<?> it = results.iterator();
        System.out.println("[SEARCH] Results:");
        while (it.hasNext()) {
            Document d = (Document) it.next();
            System.out.println("  - " +
d.getProperties().getStringValue("DocumentTitle")
                + " (" + d.get_Id() + ")");
        }
    }
}
```

STEP 5	**Compile and Run FileNetCRUD.java**	10 min

```
cd C:\Day9Labs\java

javac -cp "C:\IBM\FileNet\ContentEngine\lib\Jace.jar;^
C:\IBM\FileNet\ContentEngine\lib\log4j.jar;^
C:\IBM\FileNet\ContentEngine\lib\stax-api.jar" ^
FileNetCRUD.java

java -cp ".;C:\IBM\FileNet\ContentEngine\lib\Jace.jar;^
C:\IBM\FileNet\ContentEngine\lib\log4j.jar;^
C:\IBM\FileNet\ContentEngine\lib\stax-api.jar" ^
FileNetCRUD
```

	VERIFICATION CHECKLIST
[ ]	Compilation succeeds with no errors
[ ]	[CREATE] output shows Document created with an ID
[ ]	[UPDATE] output shows Title updated
[ ]	[SEARCH] output returns at least 1 result with title Java_API_Lab_Document_UPDATED
[ ]	Document visible in ACCE at /API_Labs/Java_Uploads with updated title

STEP 6 — Verify in ACCE (5 min)

40. Switch to ACCE browser tab
41. Navigate to: TrainingOS > Browse > API_Labs > Java_Uploads
42. Verify Java_API_Lab_Document_UPDATED appears in the folder
43. Right-click the document > Properties to confirm the updated title

[SCREENSHOT]
ACCE Browse showing Java_API_Lab_Document_UPDATED in Java_Uploads folder

	VERIFICATION CHECKLIST
[ ]	Document Java_API_Lab_Document_UPDATED visible in ACCE at /API_Labs/Java_Uploads
[ ]	Document Properties show the updated DocumentTitle
[ ]	Document has content (java_test.txt)

STEP 7 — Run CE SQL Search via ACCE Query (14 min)

In addition to the Java API search, you can run CE SQL directly from ACCE to verify results:

44. In ACCE, navigate to: Object Stores > TrainingOS > Search
45. Click SQL Search tab
46. Enter the following CE SQL:

```
SELECT Id, DocumentTitle, DateCreated FROM Document WHERE DocumentTitle LIKE
'%Java_API%'
```

47. Click Search. Verify results include the document created in Step 5.

	VERIFICATION CHECKLIST
[ ]	ACCE SQL Search returns at least 1 result for Java_API
[ ]	DateCreated shows today's date

LAB-09-04 | Event Subscription & Workflow Trigger via API

Duration: 45 Minutes | Create OnCreate event subscription and trigger workflow via PE REST API

NOTE: START STATE: Lab-09-01 through Lab-09-03 completed. TrainingWorkflow is deployed in Process Engine (from Day 7 labs). PE_ConnectionPoint is configured. WAS server is running.

Objective

In this lab you will:

- Create a Post-processing OnCreation Event Action (Process Router) in ACCE
- Create an Event Subscription to automatically launch TrainingWorkflow when a document is created in API_Labs
- Test the subscription by creating a document and verifying a workflow instance launches
- Use the Process Engine REST API to query and interact with the launched workflow

STEP 1 | Create a Process Router Event Action in ACCE | 12 min

48. In ACCE, navigate to: Object Stores > TrainingOS > Events, Actions, Processes > Event Actions
49. Click New Event Action in the right panel
50. Fill in the Event Action properties:

Display Name	`API_Lab_Workflow_Launcher`
Action Type	`Process Router`
Connection Point	`PE_ConnectionPoint`
Workflow Definition Name	`TrainingWorkflow`
Launch Step	`LaunchStep`

51. Under Data Fields Mapping, click Add to map a field:

Workflow Field	`F_Subject`
Mapping Source	`Event Object Property`
Source Property	`DocumentTitle`

52. Click OK then click Save to save the Event Action

WARNING: If TrainingWorkflow does not appear in the Workflow Definition Name dropdown, verify: (1) Process Engine service is running, (2) PE_ConnectionPoint is configured under TrainingOS, (3) TrainingWorkflow was deployed via Process Designer in Day 7 labs. If the workflow is not available, substitute any deployed workflow for this lab.

[SCREENSHOT]
ACCE Event Action configuration showing Process Router with TrainingWorkflow selected

VERIFICATION CHECKLIST

[ ]	Event Action API_Lab_Workflow_Launcher created successfully
[ ]	Action Type is Process Router
[ ]	Workflow Definition Name is TrainingWorkflow
[ ]	F_Subject field mapped to DocumentTitle property

STEP 2 — Create the OnCreation Event Subscription (10 min)

53. Navigate to: Object Stores > TrainingOS > Events, Actions, Processes > Subscriptions
54. Click New Subscription
55. Fill in the Subscription properties:

Display Name	`API_Lab_OnCreate_Subscription`
Enabled	`Yes (checked)`
Class Subscribed To	`Document (select from dropdown)`
Event	`Object Created (OnCreation)`
Timing	`Synchronous Post-Processing`
Apply To	`Instances of this class only (not subclasses, for now)`

56. Under Event Actions tab, click Add and select API_Lab_Workflow_Launcher
57. Under Scope tab, check 'Restrict to folder' and select /API_Labs

> **TIP:** Restricting the subscription to /API_Labs folder scope prevents it from launching a workflow for EVERY document created in the entire object store. In production, always scope subscriptions to the relevant folder or document class.

58. Click Save to create the subscription

[SCREENSHOT]
ACCE Subscriptions panel showing API_Lab_OnCreate_Subscription in Enabled state

	VERIFICATION CHECKLIST
[ ]	Subscription API_Lab_OnCreate_Subscription is visible in the Subscriptions list
[ ]	Status shows Enabled
[ ]	Event type is Object Created
[ ]	Event Action shows API_Lab_Workflow_Launcher

STEP 3 — Test – Create a Document to Trigger the Subscription (8 min)

59. In ACCE, navigate to: TrainingOS > Browse > API_Labs > REST_Uploads
60. Right-click REST_Uploads > New Document

Document Title	`Subscription_Test_Document_01`
Content	`Upload any small file from C:\Day9Labs\rest\test_document.txt`
Class	`Document`

61. Click Finish to create the document
62. Expected: The document is created AND a workflow instance is launched in the background

TIP: If you see an error during document creation such as 'workflow launch failed', check: (1) Process Engine service is running (check services.msc), (2) PE_ConnectionPoint status in ACCE, (3) WAS SystemOut.log for PE connection errors.

STEP 4 — Verify Workflow Instance via ACCE Workflow Monitor (5 min)

63. In ACCE, navigate to: Object Stores > TrainingOS > Events, Actions, Processes > Work Item Queues
64. Alternatively, open the Work Item list: Workflows > TrainingWorkflow > Active Instances
65. Look for a workflow instance with subject matching: Subscription_Test_Document_01

[SCREENSHOT]
ACCE Workflow active instances showing Subscription_Test_Document_01 workflow

	VERIFICATION CHECKLIST
[ ]	Workflow instance created with F_Subject = Subscription_Test_Document_01
[ ]	Instance status is Active or In Progress
[ ]	DateLaunched shows today's date and current time

STEP 5 — Query Workflow Instances via PE REST API (10 min)

Use curl to query the Process Engine REST API for active workflow instances:

```
# Get active queue items from the workflow queue
curl -u ceadmin:ceadmin ^
     -H "Accept: application/json" ^
     "http://localhost:9080/pe-
rest/v1/workobjects/queueitems?queueName=TrainingWorkflow_Queue" ^
     > queue_items.json

type queue_items.json
```

NOTE: The queue name depends on how TrainingWorkflow was designed in Day 7. If you do not know the queue name, use the following endpoint to list all queues: curl -u ceadmin:ceadmin -H "Accept: application/json" "http://localhost:9080/pe-rest/v1/queues"

66. Alternatively, launch a new workflow instance directly via PE REST API:

```
curl -u ceadmin:ceadmin ^
     -X POST ^
     -H "Content-Type: application/json" ^
     -H "Accept: application/json" ^
     -d
"{\"wfDefinition\":\"TrainingWorkflow\",\"data\":{\"F_Subject\":\"REST_Triggered_Workflow_
01\"}}" ^
     "http://localhost:9080/pe-rest/v1/workobjects" ^
     > launch_result.json

type launch_result.json
```

	VERIFICATION CHECKLIST
[ ]	PE REST API returns list of queue items OR launch result JSON
[ ]	launch_result.json contains a workObjectNumber for the new workflow instance
[ ]	New workflow instance visible in ACCE Workflow Monitor

Day 9 Lab Completion Summary

Lab	Title	Skills Practiced	Status
LAB-09-01	Environment & Prerequisites	Verify CE, CMIS, curl, JARs	[ ] Complete
LAB-09-02	Upload via REST API	curl, CMIS Browser Binding	[ ] Complete
LAB-09-03	Java CEI CRUD Operations	Java API, compile, run	[ ] Complete
LAB-09-04	Event Subscription & Workflow	ACCE subscription, PE REST	[ ] Complete